Advertising Strategy Development: Textual Analysis of User Reviews

25SS Text Technology Project, IMS, Universität Stuttgart
Presented by: Qianyue Li, Qianru Qian, Xuetong Yang
Date: 16.07.2025

Project Overview

Project Story

We are a marketing company interested in comparing customer reviews on two competing fast-food brands — Burger King and McDonald’s — to uncover user preferences and guide data-driven advertising strategies.

Project Goals

Extract insights from real user reviews

Compare brand perception across specific service dimensions

Support marketing decisions with structured, explainable data

Project Outline

Tools used in each section

Collect: JSON -> excel, manual annotation per a written guideline

Prepare: XML, XML schema, Xpath, Xpointer (extension)

Access: Database, SQL, Neo4j, RDF (extension)

Collect

- Extract user reviews from the Yelp Open Dataset (JSON format)

- Filter reviews from the target brands: Burger King and McDonald’s
  - Step1: identify top 5 most-reviewed store locations per brand
  - Step2: From each store, select the top 10 reviews based on Yelp's “Most Useful” user votes
  - ✔ Total dataset: 2 brands × 5 stores × 10 reviews = 100 reviews

- Annotate sentiment polarity , focusing on four aspects: **Service, Food quality, Environment, and Price**.
- Each aspect is tagged with a sentiment judgement: **good/neutral/bad/N/A**, associated **keywords** as supporting evidence

Prepare: XML Schema & Data Validation

Schema Design Summary

Root: **<reviews>** → has multiple <review> elements

<review> has
- attributes: review_id, brand
- elements: text, annotations

<text> stores original review
<annotations> contains 4 aspects: service, food quality, environment, price
- **<aspect>** has:
  - **<rating>** : good, bad, neutral, n/a
  - **<keyword>**: explaining the rating (e.g, “friendly”, “rude”...)

Schema Snippet – Structure of <review> Element

```
<xs:element name="review">
  <xs:complexType>
    <xs:sequence>
      <xs:element name="text" type="xs:string"/>
      <xs:element ref="annotations"/>
    </xs:sequence>
    <xs:attribute name="review_id" type="xs:string" use="required"/>
    <xs:attribute name="brand" type="brandType" use="required"/>
  </xs:complexType>
</xs:element>
```

Prepare: XPath and XPointer as extension

XPath to inspect the most important information & XPointer to precisely locate and reference keywords within the review text:

Step 1 Apply **XPath** to find the most important aspect (service) —> extract related keywords

- Extract all valid review sentiments (not n/a) and their counts for each brand to identify the focus of analysis.
- Get all the keywords with positive/negative sentiments on service aspect for each brand

```
xpath -e 'count(//review[@brand="Burger King"]//service[rating!="n/a"])' reviews.xml
…
xpath -e '//review[@brand="Burger King"]//service[rating="good"]/keyword' reviews.xml > BK_service_good.txt
…
```

Percentages may not sum to 100% due to rounding.

Prepare: XPath and XPointer as extension

Step2 Precise Keyword Anchoring with **XPointer**

- Locate each keyword’s position in the review text

```
<service>
    <rating>good</rating>
    <keyword start="81" end="92" xlink:href="#xpointer(string-range(//review[@review_id='A9OYKEpFpqX_YrESn5d8XQ']/text,'efficiently'))">efficiently</keyword>
    <keyword start="102" end="112" xlink:href="#xpointer(string-range(//review[@review_id='A9OYKEpFpqX_YrESn5d8XQ']/text,'good humor'))">good humor</keyword>
    <keyword start="210" end="218" xlink:href="#xpointer(string-range(//review[@review_id='A9OYKEpFpqX_YrESn5d8XQ']/text,'gracious'))">gracious</keyword>
</service>
```

Step3 Keyword Processing & Grouping

- Store keywords in a Pandas Dataframe and perform tokenization & lemmatization
- Group keywords in Service into different aspects *(e.g., Staff attitude, Timeliness)* and get the subjectivity of each service review through TextBlob

Access

Step1 Create SQL tables

- reviews (review_id, brand, text);
- ratings(review_id, brand, aspect, rating);
- keywords (keyword_id, review_id, aspect, rating, keyword, xpointer);
- service_subaspects (id, review_id, keyword, sub_aspect , subjectivity)

Step2 Import XML Data into SQL

- Parse Dataframe with Python and insert into SQL tables

Access

Step3 SQL query examples

a) Check the positive and negative percentages of the two brands in 4 aspects

```
SELECT brand,aspect,
COUNT(*) FILTER (WHERE rating = 'good') AS positive_count,
ROUND(COUNT(*) FILTER (WHERE rating = 'good') * 100.0 / COUNT(*), 1) AS positive_percentage,
COUNT(*) FILTER (WHERE rating = 'bad') AS negative_count,
ROUND(COUNT(*) FILTER (WHERE rating = 'bad') * 100.0 / COUNT(*), 1) AS negative_percentage
FROM ratings
GROUP BY brand, aspect
ORDER BY positive_percentage DESC;
```

brand	aspect	positive_count	positive_percentage	negative_count	negative_percentage
Burger King	service	16	32.0	28	56.0
Burger King	foodQuality	9	18.0	17	34.0
Burger King	environment	6	12.0	3	6.0
Burger King	price	4	8.0	0	0.0
McDonald's	environment	4	8.0	12	24.0
McDonald's	service	3	6.0	41	82.0
McDonald's	foodQuality	1	2.0	14	28.0
McDonald's	price	1	2.0	1	2.0

Access

Step3 SQL query examples

b) Identify top 5 negative keywords per brand

```
SELECT brand, keyword, count
FROM (
   SELECT r.brand, k.keyword, COUNT(*) AS count, ROW_NUMBER() OVER (PARTITION BY r.brand ORDER BY COUNT(*) DESC) AS rank FROM keywords k
   JOIN reviews r ON k.review_id = r.review_id
   WHERE k.rating = 'bad'
   GROUP BY r.brand, k.keyword
) ranked
WHERE rank <= 5
ORDER BY brand, count DESC;
```

brand	keyword	count
Burger King	rude	2
Burger King	raw	2
Burger King	food is horrible	1
Burger King	whopper Jr had no cheese on it and full of tomatoes.	1
Burger King	salty, soggy	1
McDonald's	safety	1
McDonald's	People will sit in the drive-thru for over 30 minutes	1
McDonald's	take really long	1
McDonald's	beggars hanging around the parking lot	1
McDonald's	the floor needed to be swept	1

Access

Step4 SQL full text search examples

a) Create an index for reviews’ text to speed up searches

```
CREATE INDEX idx_reviews_text_fts ON reviews USING GIN(to_tsvector('english', text));
```

b) Find the review that mentions "'slow" or "wait'" and see where exactly it appears in the original review.

```
SELECT brand,aspect,
SELECT k.review_id, r.brand, k.keyword, k.xpointer, r.text, ts_rank(to_tsvector('english', r.text), to_tsquery('slow & wait')) AS rank
FROM keywords k
JOIN reviews r ON k.review_id = r.review_id
WHERE to_tsvector('english', r.text) @@ to_tsquery('slow & wait')
                    AND (k.keyword ILIKE '%slow%' OR k.keyword ILIKE '%wait%')
ORDER BY rank DESC, k.review_id DESC
LIMIT 10;
```

Access

Step4 SQL full text search examples

c) Records with the highest subjective STAFF_ATTITUDE and the word friendly/helpful/kind in the text

```
SELECT s.review_id, s.keyword, s.sub_aspect, s.subjectivity, ts_rank(to_tsvector('english', r.text), to_tsquery('friend:* | helpful:* | kind:*')) AS rank
FROM Service_subaspects s
JOIN reviews r ON s.review_id = r.review_id
WHERE s.sub_aspect = 'STAFF_ATTITUDE'
AND to_tsvector('english', r.text) @@ to_tsquery('friend:* | helpful:* | kind:*')
AND s.keyword ILIKE ANY (ARRAY['%friendly%', '%helpful%', '%kind%'])
ORDER BY s.subjectivity DESC, rank DESC
LIMIT 10;
```

review_id	keyword	sub_aspect	subjectivity	rank
2axjDKlP1M1iNPv-bQVJKw	friendly	STAFF_ATTITUDE	0.65	0.020264236
rKC3_jFuU4UT6_ZBLENxnQ	friendly	STAFF_ATTITUDE	0.65	0.020264236
JG6-FvhqDbR541own3JE-w	friendly	STAFF_ATTITUDE	0.65	0.020264236
JG6-FvhqDbR541own3JE-w	friendly	STAFF_ATTITUDE	0.5	0.020264236
JG6-FvhqDbR541own3JE-w	friendly	STAFF_ATTITUDE	0.5	0.020264236
2axjDKlP1M1iNPv-bQVJKw	friendly	STAFF_ATTITUDE	0.5	0.020264236
rKC3_jFuU4UT6_ZBLENxnQ	friendly	STAFF_ATTITUDE	0.5	0.020264236
JG6-FvhqDbR541own3JE-w	friendly	STAFF_ATTITUDE	0.5	0.020264236
2axjDKlP1M1iNPv-bQVJKw	friendly	STAFF_ATTITUDE	0.5	0.020264236
rKC3_jFuU4UT6_ZBLENxnQ	friendly	STAFF_ATTITUDE	0.5	0.020264236

Access - Knowledge Graph (Neo4j & Cypher)

```
// create nodes
MERGE (b:Brand {name: $brand_name})

MERGE (r:Review {reviewID: $review_id})
ON CREATE SET r.subjectivity = $subjectivity_val

MERGE (s:Sentiment {type: $sentiment_type})

MERGE (a:Aspect {name: $aspect_name})

MERGE (k:Keyword {word: $keyword_word})

// link nodes
MERGE (b)-[:HAS_REVIEW]->(r)
MERGE (r)-[:EXPRESSES_SENTIMENT]->(s)
MERGE (r)-[:RELATES_TO_ASPECT]->(a)
MERGE (r)-[:CONTAINS_KEYWORD]->(k)
```

Nodes:

- Brand
- Review
- Sentiment
- Aspect
- Keyword

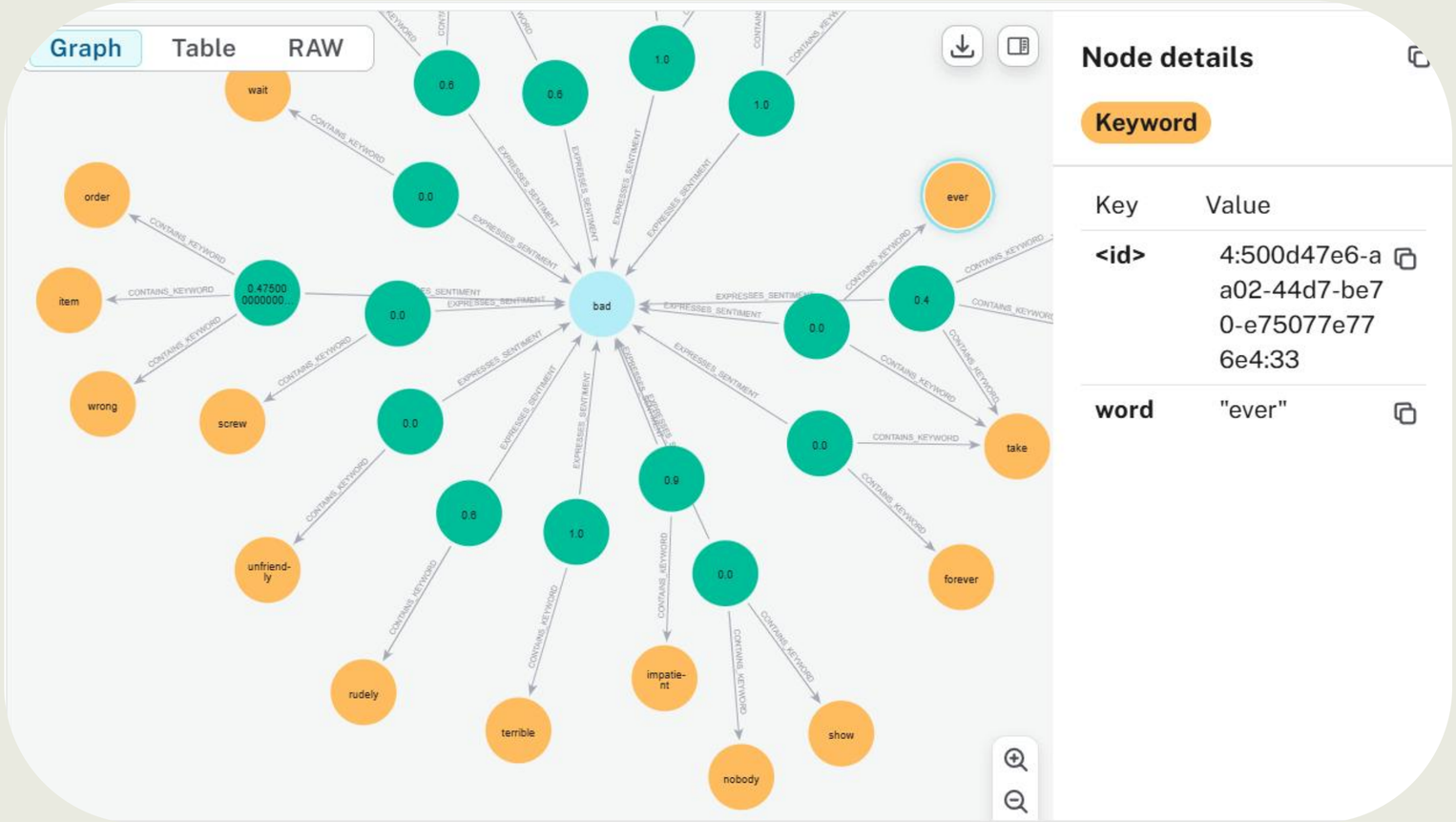
Edges:

- HAS_REVIEW
- EXPRESSES_SENTIMENT
- RELATES_TO_ASPECT
- CONTAINS_KEYWORD

Value:

- Subjectivity (for Review)

Access - Knowledge Graph (Neo4j & Cypher)



Infer the relational knowledge web for sentiment, reviews (& their subjectivity) and the keywords contained in each review.

```
MATCH p=()<-[:CONTAINS_KEYWORD]-(r:Review)-[:EXPRESSES_SENTIMENT]->()
RETURN p
LIMIT 25;
```

Access - Knowledge Graph (Neo4j & Cypher)

	Brand	Aspect	AverageSubjectivity	MinSubjectivity	MaxSubjectivity
1	"BK"	"ORDER_ACCURACY"	0.45833333333333337	0.0	0.9
2	"BK"	"OTHER"	0.40920000000000006	0.0	1.0
3	"BK"	"STAFFING_LEVELS"	0.03333333333333333	0.0	0.1
4	"BK"	"STAFF_ATTITUDE"	0.4617391304347826	0.0	1.0
5	"BK"	"TIMELINESS"	0.23547008547008547	0.0	0.6111111111111112
6	"MC"	"ORDER_ACCURACY"	0.30000000000000004	0.0	0.9
7	"MC"	"OTHER"	0.2143990929705215	0.0	1.0
8	"MC"	"STAFFING_LEVELS"	0.0	0.0	0.0
9	"MC"	"STAFF_ATTITUDE"	0.5272727272727273	0.0	1.0
10	"MC"	"TIMELINESS"	0.24310185185185187	0.0	1.0

MATCH (b:Brand)-[:HAS_REVIEW]->(r:Review)

WHERE b.name **IN** ['BK', 'MC']

RETURN b.name **AS** Brand, a.name **AS** Aspect,
avg(r.subjectivity) **AS** AverageSubjectivity,
min(r.subjectivity) **AS** MinSubjectivity,
max(r.subjectivity) **AS** MaxSubjectivity

ORDER BY Brand, Aspect

Infer and compare the average/min/max subjectivity of reviews pointing to each aspect of service for two brands respectively: ***staff attitude is most related in terms of subjectivity***

Access extension - Knowledge Graph (RDF & SPARQL)

```
@prefix data: <http://example.org/reviews/data/> .
@prefix wd: <http://www.wikidata.org/entity/>
@prefix ex: <http://example.org/reviews/ontology#> .
@prefix schema1: <http://schema.org/> .
@prefix xsd: <http://www.w3.org/2001/XMLSchema#> .
```

```
data:review_47 a schema1:Review ;
    ex:containsKeyword data:keyword_easy ;
    ex:expressesSentiment data:sentiment_good ;
    ex:relatesToAspect data:aspect_OTHER ;
    ex:withSubjectivity "0.8333333333333334"^^xsd:float ;
    schema1:itemReviewed wd:Q177054 .
```

```
wd:Q177054 a schema1:Brand ;
    schema1:name "Burger King" .
```

RDF (Resource Description Framework)

- similar to Neo4J
- Global uniqueness: uses URI for entities
- emphasize more on the cross database compatibility and interoperability

SPARQL

- query language for rdf

Access extension - Knowledge Graph (RDF & SPARQL)

brandLabel	totalAssets	totalNetIncome	goodReviewsPct	badReviewsPct	neutralReviewsPct
1"Burger King"@en	"1970000000"^^<http...	"117700001"^^<http://www...	"37.3134328358208955223...	"62.686567164179104477...	"0.0"^^<http://www.w3.org/2001/XM...
2"McDonald's"@en	"23182600000"^^<ht...	"6177400000"^^<http://ww...	"4.0"^^<http://www.w3.org/2001/...	"85.333333333333333333...	"10.666666666666666666666...

```
SELECT
    ?brandLabel
    (SAMPLE(?assets) AS ?totalAssets)
    (SAMPLE(?netIncome) AS ?totalNetIncome)
    (COALESCE(?goodPct, 0) AS ?goodReviewsPct)
    ...
WHERE {
  VALUES ?brandURI { wd:Q177054 wd:Q38076 }

# --- Local Graph: Calculate Review Percentages ---
  {... }
```

```
# --- Query to Wikidata for Financials ---
  SERVICE <https://query.wikidata.org/sparql> {
    ?brandURI rdfs:label ?brandLabel .
    FILTER(LANG(?brandLabel) = "en")

    OPTIONAL { ?brandURI wdt:P2139 ?assets . }
    OPTIONAL { ?brandURI wdt:P2295 ?netIncome . }
  }
}
GROUP
BY ?brandLabel ?assets ?netIncome ?goodPct ?badPct ?neutralP
ct
ORDER BY ?brandLabel
```

Strategic Takeaways

Key Findings

- Burger King receives significantly higher positive feedback on Service and Food Quality.
- McDonald’s is criticized more often for service, while Burger King benefits from staff friendliness.
- Staff attitude stands out as the most emotionally subjective aspect.

Marketing Implications

- Favor Burger King as the primary ad client in campaigns.
- Highlight service quality especially staff friendliness in advertising.
- Deploy campaigns on platforms where emotional engagement drives behavior (e.g., Instagram, TikTok).

Challenges

Data Quality & Selection

- Yelp reviews vary widely in tone, length, and clarity
- Some reviews may lack sentiment polarity
- Solution: Use top 10 “most useful” reviews only

Annotation Complexity

- One review may contain multiple mixed sentiments
- Solution: Create clear annotation guideline + peer review annotations

Schema Design & Validation

- Ensuring schema captures flexibility while remaining strict enough to validate

Data Transformation: XML → SQL

- Requires flattening hierarchical structure into relational tables

Access to Public Knowledge Graph Database

- Tried DBpedia -> can only fetch metadata from Mcdonalds’ page but not info on assets, etc.
- Solution: use Wikidata instead, but some information like the number of locations are missing

References

Apache Jena Fuseki sparql server
https://jena.apache.org/documentation/fuseki2/

IMS – Text Technology Course Materials
University of Stuttgart, Summer Semester 2025

Neo4j Graph Database
https://neo4j.com/

W3C XML Schema Definition Language
https://www.w3.org/TR/xmlschema11-1/

W3C XPath and XPointer Specification
https://www.w3.org/TR/xptr/

Wikidata database
https://www.wikidata.org/

Yelp Open Dataset
https://www.yelp.com/dataset

Thank you